



Operating Systems & Networks CTEC1004C

Lecture - 1 : Number System

Dr. Sameer Jha, SFHEA

Faculty of Computing

De Montfort University Cambodia

Index

- Bases and Conversions
- Types of Number Systems
- Memory & Storage Capacities
- Binary in Memory, Storage, and CPU Operations
- Hexadecimal in Memory Addresses, Colours, and MAC Addresses
- Number Systems in Networking: IPv6 & Subnets
- Hex in File Formats, ASCII Encoding, and Cryptography

Introduction

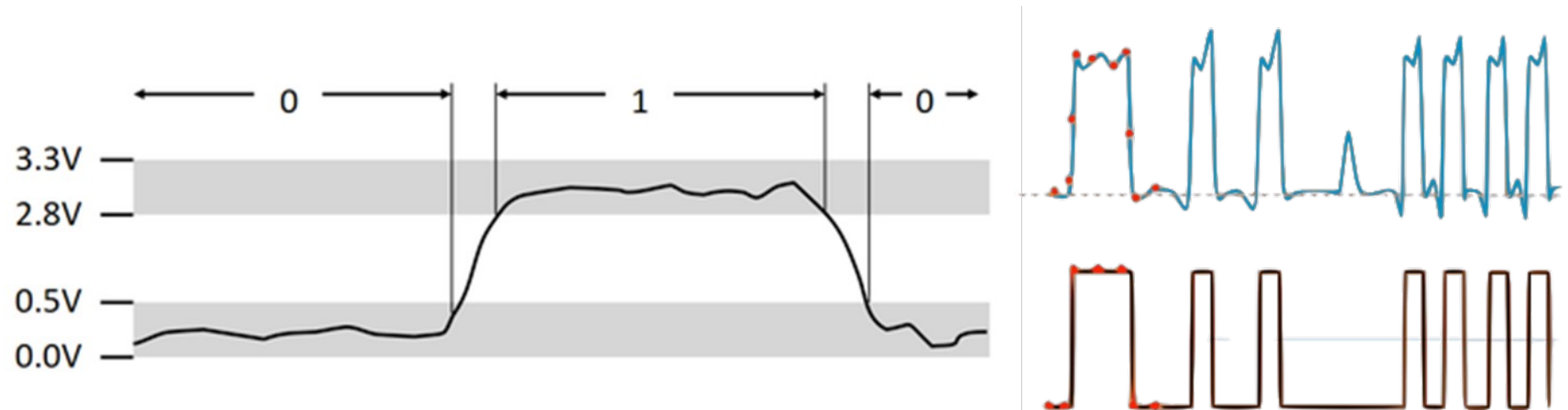
- Binary numbers -All computers work with 0's and 1's so it is like learning alphabets before learning English
- Other Bases – In CompSci and SoftEng, people use other bases as well like hexadecimal, we need to know the logic behind bases, and conversions between them.

Introduction (Cont.)

- Electronic and digital systems use various number systems such as Decimal, Binary, Hexadecimal and Octal, which are essential in computing.
- Binary (base-2) is the foundation of digital systems.
- Hexadecimal (base-16) and Octal (base-8) are commonly used to simplify the representation of binary data.
- The Decimal system (base-10) is the standard system for everyday calculations.
- Other number systems like Duodecimal (base-12), are less commonly used but have specific applications in certain fields.

Bases and Conversions

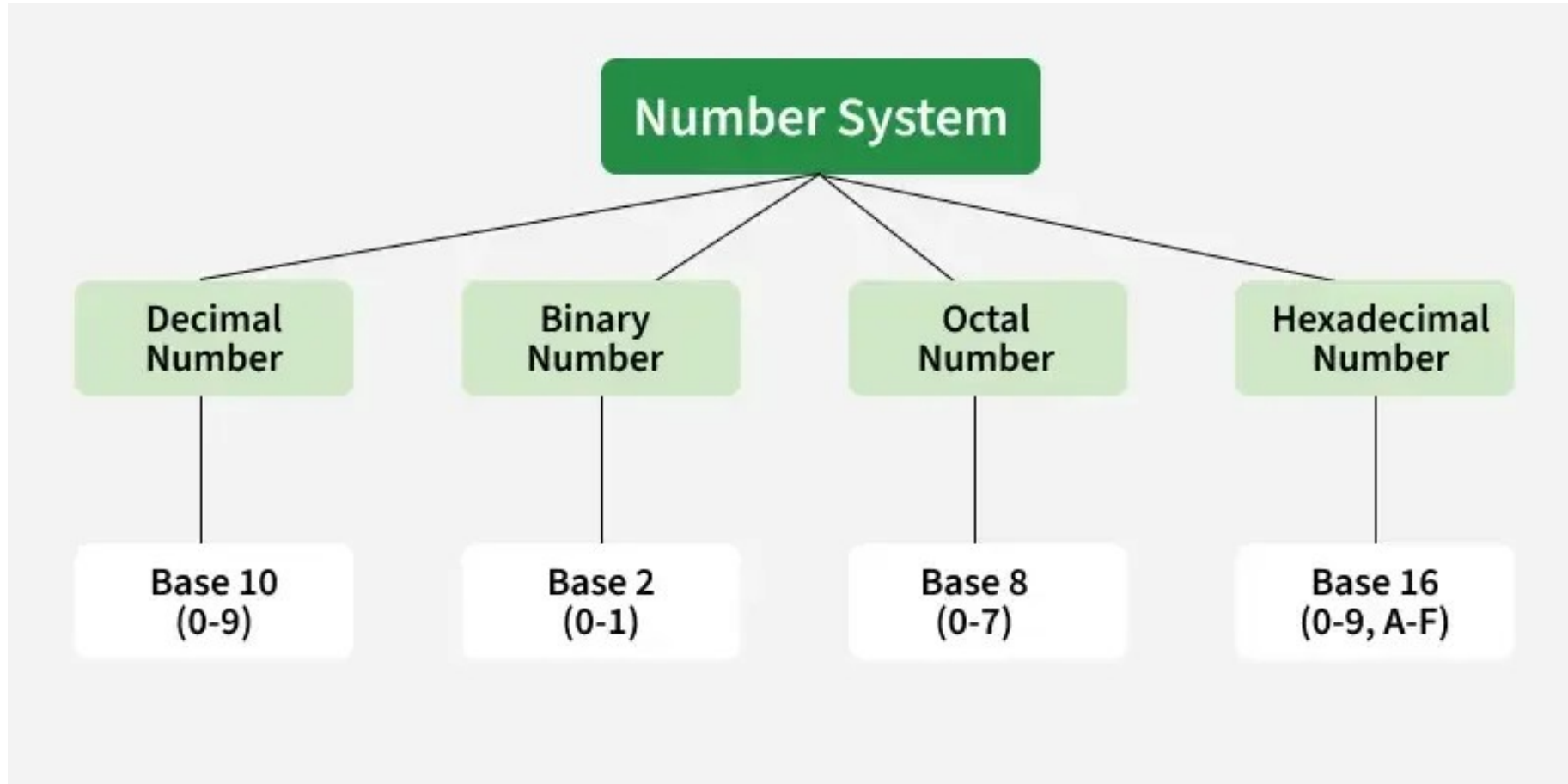
- On a hardware level, numbers are represented using states of physical objects
- It is easy to distinguish 2 states (e.g. high/low voltage, direction of a magnetic field), but difficult to distinguish more than 2 states



Bases we will study

- Binary: Base 2
 - Octal: Base 8
 - Decimal: Base 10
 - Hexadecimal: Base 16
-
-and converting between them.
-
- What are bases? – Definition: A number base is the number of digits or combination of digits that a system of counting uses to represent numbers. E.g Binary uses 1,0 , so Base 2.

Types of Number Systems



How do Numbers work?

- Positionally.
- Case in point – Base 10, you know the “normal” numbers we use everyday.
- $101 = 1 \times 10^2 + 0 \times 10^1 + 1 \times 10^0$ -- for base 10
- Each digit in the number is multiplied by 10 raised to a power corresponding to that digit's position
- Another one – Base 8
- $63 = 6 \times 8^1 + 3 \times 8^0$ -- This time, it is multiplied by 8, since the base is 8.

Fractions in Base 10

- The same principle holds for decimal fractions, but using negative powers of 10

For example –

- The decimal fraction 0.256 stands for 2 tenths plus 5 hundredths plus 6 thousandths: $0.256 = (2 * 10^{-1}) + (5 * 10^{-2}) + (6 * 10^{-3})$
- A number with both an integer and fractional part has digits raised to both positive and negative powers of 10
- **Most significant digit** The leftmost digit (carries the highest value)
- **Least significant digit** The rightmost digit (carries the least value)

Positional Interpretation of a Decimal Number

• 472.256

4	7	2	2	5	6
100s	10s	1s	tenths	hundredths	thousandths
10^2	10^1	10^0	10^{-1}	10^{-2}	10^{-3}
position 2	position 1	position 0	position -1	position -2	position -3

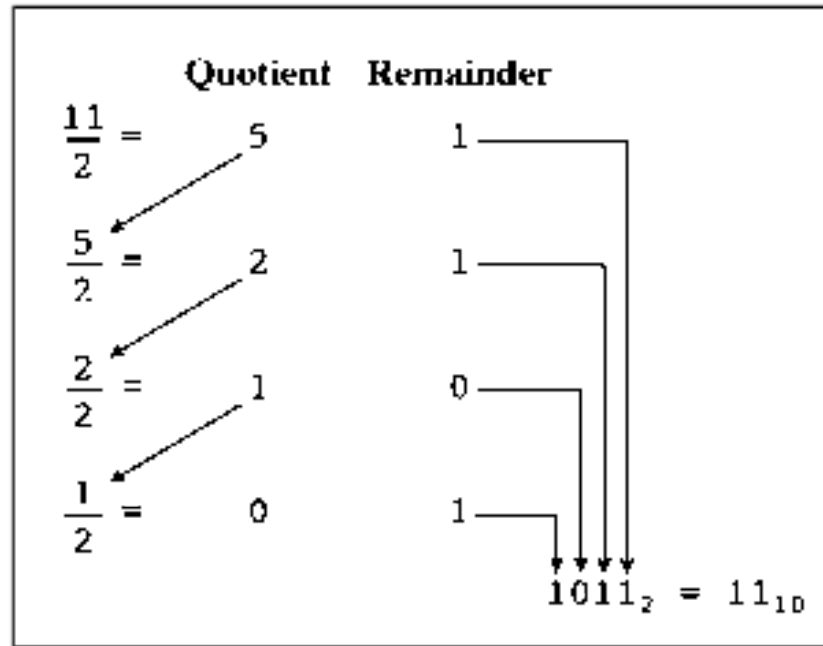
Converting Binary to Decimal

- Binary notation to decimal notation:
 - Multiply each binary digit by the appropriate power of 2 and add the results
 - $1001.101 = 2^3 + 2^0 + 2^{-1} + 2^{-3} = 8 + 1 + 0.5 + 0.125 = 9.625_{10}$

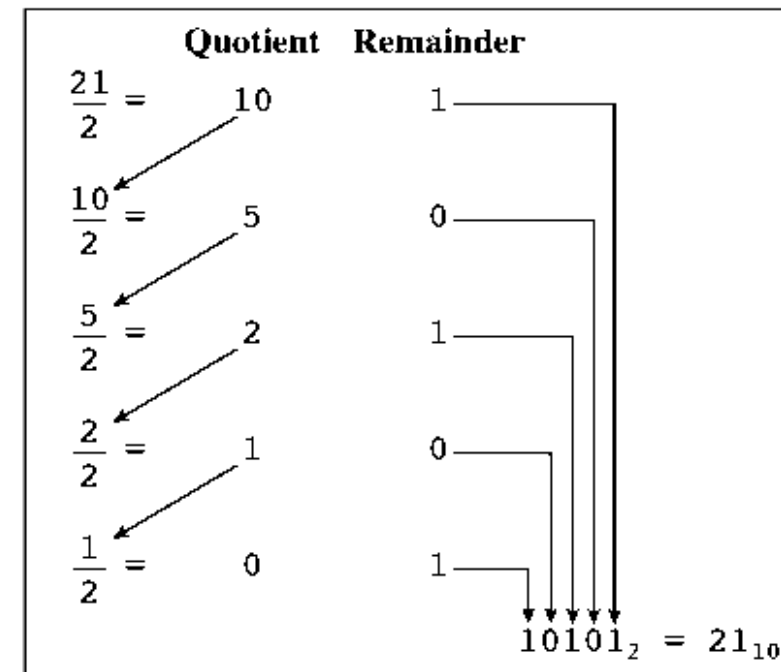
Converting Decimal to Binary: Integers

- Integer and fractional parts are handled separately
- For the integer part:
 1. Divide the decimal number by 2
 - If there IS a remainder the rightmost digit will be a 1
 - If there is NO remainder, the rightmost digit will be a 0.
 2. Then repeat the process, moving one digit to the left each time, until you have divided down to 1.

Decimal to Binary: Integers (Examples)



(a) 11_{10}



(b) 21_{10}

Decimal to Binary: Fractions

1. Multiply the fractional part by 2
 - If the result is smaller than 1, the next most significant binary digit is 0
 - Otherwise, the next binary digit is 1
2. Take the results' fractional part and repeat the process
 - Either until the fractional part is 0
 - Or until you reach the desired number of fractional digits

Decimal to Binary: Fraction(Examples)

Product	Integer Part	0.01 ₂
0.25 × 2 = 0.5	0	↑↑
0.5 × 2 = 1.0	1	

(b) $0.25_{10} = 0.01_2$ (exactly)

Product	Integer Part	0.110011 ₂
0.81 × 2 = 1.62	1	↑↑↑↑↑↑
0.62 × 2 = 1.24	1	
0.24 × 2 = 0.48	0	
0.48 × 2 = 0.96	0	
0.96 × 2 = 1.92	1	
0.92 × 2 = 1.84	1	

(a) $0.81_{10} = 0.110011_2$ (approximately)

Hexadecimal - Base 16.

Decimal (base 10)	Binary (base 2)	Hexadecimal (base 16)
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	0001 0000	10
17	0001 0001	11
18	0001 0010	12
31	0001 1111	1F
100	0110 0100	64
255	1111 1111	FF
256	0001 0000 0000	100

Hexadecimal Notation: Base 16

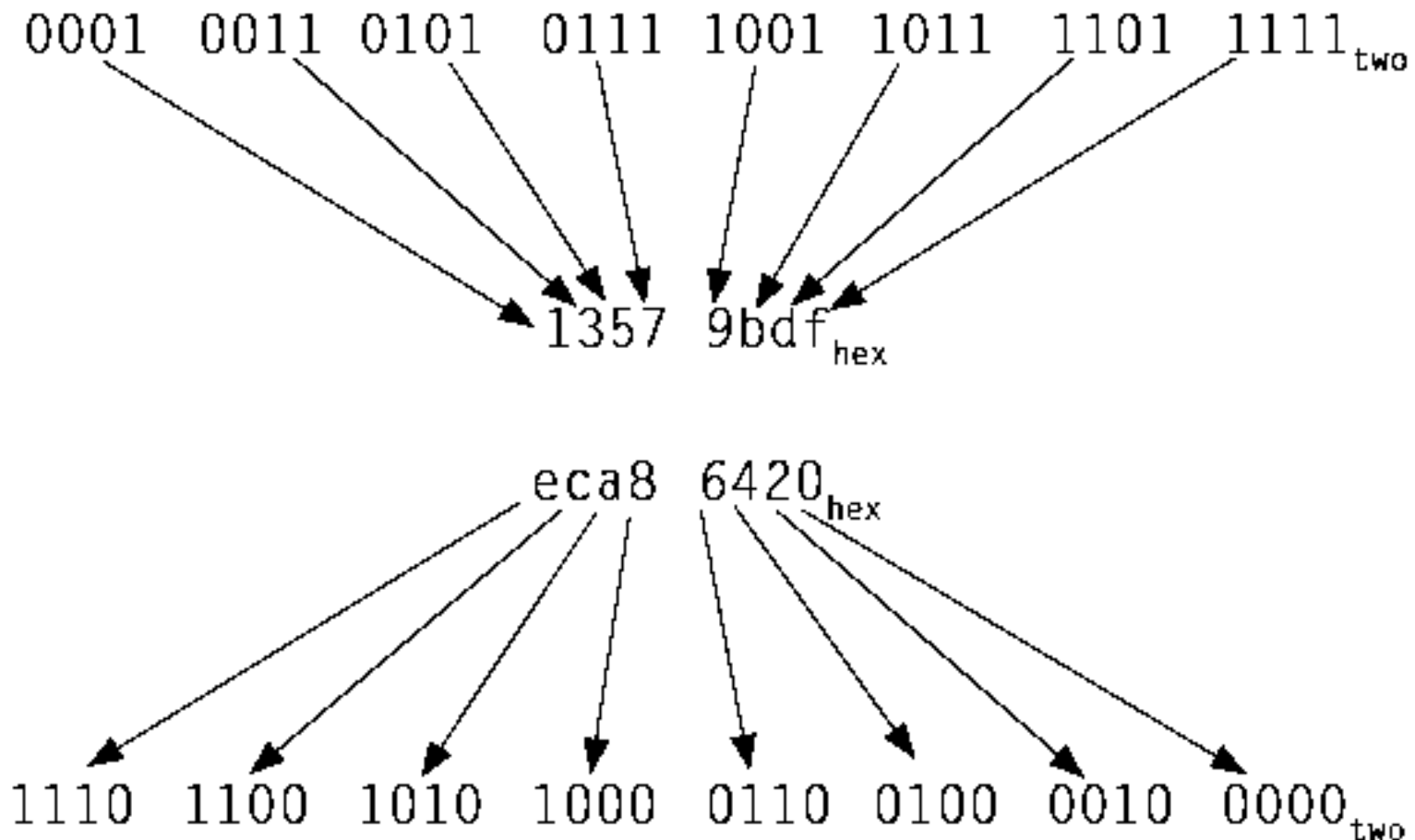
- Binary digits are grouped into sets of four bits, called a nibble
- Each possible combination of four binary digits is given a symbol, as follows:

0000 = 0	0100 = 4	1000 = 8	1100 = C
0001 = 1	0101 = 5	1001 = 9	1101 = D
0010 = 2	0110 = 6	1010 = A	1110 = E
0011 = 3	0111 = 7	1011 = B	1111 = F

- Because we use 16 symbols, the notation is called hexadecimal and the 16 symbols are the hexadecimal digits
- For example:

$$\begin{aligned} 2C_{16} &= (2_{16} * 16^1) + (C_{16} * 16^0) \\ &= (2_{10} * 16^1) + (12_{10} * 16^0) = 44 \end{aligned}$$

Binary to Hexadecimal and back



Hexadecimal Notation

- Reasons for using hexadecimal notation are:
 - More compact than binary notation
 - In most computers, binary data occupy some multiple of 4 bits, and hence some multiple of a single hexadecimal digit
 - It is extremely easy to convert between binary and hexadecimal notation.

Memory & Storage Capacities

- Computer RAM stores data as binary values, where each memory cell holds one bit. Because memory addressing is binary, storage capacities naturally scale in **powers of 2**

Unit	Exact Size (Bytes)	Binary Equivalent
1 Kilobyte (KB)	1,024	2^{10} bytes
1 Megabyte (MB)	1,048,576	2^{20} bytes
1 Gigabyte (GB)	1,073,741,824	2^{30} bytes
1 Terabyte (TB)	~1.1 trillion	2^{40} bytes

The "Missing Storage" Phenomenon

Why does a "500 GB" hard drive appear as only ~465 GB in Windows?

Manufacturers advertise in **decimal gigabytes** (1 GB = 10^9 = 1,000,000,000 bytes). Operating systems report in **binary gigabytes** (1 GiB = 2^{30} = 1,073,741,824 bytes).

$500,000,000,000 \div 1,073,741,824 \approx 465.66$ GiB.

CPU Operations & Logic Gates

- Every arithmetic and logical operation a CPU performs is implemented using **logic gates** - physical circuits that operate on binary inputs to produce binary outputs.

AND	Outputs 1 only if both inputs are 1.
OR	Outputs 1 if at least one input is 1.
XOR	Outputs 1 if inputs are different (used for addition).
NOT	Inverts the input (1 becomes 0, 0 becomes 1).

- These fundamental gates combine to create complex circuits like **Full Adders**, which perform binary arithmetic inside the CPU's Arithmetic Logic Unit (ALU).

$$\begin{array}{r} 1011 \quad (11 \text{ in decimal}) \\ + 0110 \quad (6 \text{ in decimal}) \\ \hline 10001 \quad (17 \text{ in decimal}) \end{array}$$

- Carry propagation in binary addition (like the 1 carried over to the 5th bit above) is handled physically by chaining logic gates together.

Hexadecimal in Memory Addresses

- When a program runs, the operating system assigns it a region of memory. Every memory location has a unique address.
- Addresses are expressed in hexadecimal because they are fixed-width binary values. A 32-bit system uses **8 hex digits** per address; a 64-bit system uses up to **16 hex digits**.
- When you see a crash report, every address shown is hexadecimal.

WINDOWS PROCESS MEMORY MAP

```
0x00400000 // Start of 32-bit executable code
0x7FFFFFFF // Upper boundary of user-mode
0xC0000000 // Start of kernel memory space
```

Hexadecimal in Colours

- In web design and digital imaging, colours are represented using the **RGB model**: three components (Red, Green, Blue).
- Each component ranges from 0 to 255, which is exactly one byte, or two hexadecimal digits.

HEX COLOUR BREAKDOWN

Colour: #3A7BD5

Red = $3A_{16} = 58_{10}$

Green = $7B_{16} = 123_{10}$

Blue = $D5_{16} = 213_{10}$

Colour	Hex Code	R	G	B
 Red	#FF0000	255	0	0
 Green	#00FF00	0	255	0
 Blue	#0000FF	0	0	255
 White	#FFFFFF	255	255	255

Hexadecimal in MAC Addresses

- A **MAC (Media Access Control) address** uniquely identifies a network interface at the hardware level.
- It is a **48-bit (6-byte)** value, always written in hexadecimal, separated by colons or hyphens.
- MAC addresses are used by Ethernet switches to forward frames within a

MAC ADDRESS STRUCTURE

Format: **XX:XX:XX:XX:XX:XX**

Example: **00:1A:2B:3C:4D:5E**

// First 3 bytes (00:1A:2B)

OUI (Manufacturer ID)

// Last 3 bytes (3C:4D:5E)

Device-specific ID

Number Systems in Networking: IPv6 & Subnets

- IPv4 addresses (32-bit, e.g., 192.168.1.1) are exhausted. IPv6 uses 128-bit addresses, providing 2^{128} ($\approx 3.4 \times 10^{38}$) unique addresses.
 - Writing 128 bits in binary is unreadable. Instead, IPv6 addresses are written as **hexadecimal digits**, see
- FULL IPV6 ADDRESS EXAMPLE

2001:0DB8:85A3:0000:0000:8A2E:0370:7334
- **Shorthand Rules:**
 1. Leading zeros in a group can be omitted.
 2. Consecutive all-zeroes can be replaced with :: (only once per address)
- COMPRESSED IPV6 ADDRESS

2001:DB8:85A3::8A2E:370:7334

Without hexadecimal notation, managing modern network infrastructure would be virtually impossible for human engineers.

Number Systems in Networking: IPv6 & Subnets (Cont.)

- IP subnetting relies entirely on **binary arithmetic**. A subnet mask is a 32-bit binary number that separates the network identifier from the host identifier.
- The network address is found by performing a **bitwise AND** operation between the IP address and

Component	Decimal	Binary Representation
IP Address	192.168.1.100	11000000.10101000.00000001.01100100
Subnet Mask	255.255.255.0	11111111.11111111.11111111.00000000
Network (AND)	192.168.1.0	11000000.10101000.00000001.00000000

Number Systems in Networking: IPv6 & Subnets (Cont.)

- **CIDR Notation (Classless Inter-Domain Routing):**

Instead of writing the full mask, CIDR counts the number of consecutive 1 bits in the mask.

CIDR EXAMPLES

`/24 = 255.255.255.0 (24 ones, 8 zeros)`

`/26 = 255.255.255.192 (26 ones, 6 zeros)`

`/28 = 255.255.255.240 (28 ones, 4 zeros)`

Hex in ASCII Encoding

- Every character typed is stored as a binary number. The **ASCII** standard assigns a unique 8-bit number to each character, commonly represented in hexadecimal

Char	Decimal	Hexadecimal	Binary
A	65	0x41	0100 0001
a	97	0x61	0110 0001
0	48	0x30	0011 0000
Space	32	0x20	0010 0000

- **Note:** Uppercase 'A' (0x41) and lowercase 'a' (0x61) differ by exactly 0x20 (bit 5). Toggling this bit switches case.

Hex in File Formats

- Operating systems identify file types not by their extension, but by the first few bytes of the file - the **magic number** (file signature), always expressed in hex.

File Type	Magic Number (Hex)	ASCII Equivalent
JPEG Image	FF D8 FF	(non-printable)
PNG Image	89 50 4E 47	.PNG
PDF Document	25 50 44 46	%PDF
Windows EXE	4D 5A	MZ

- Security analysts use hex editors to inspect magic numbers when detecting malware disguised with false extensions.*

Hex in Cryptography

- Modern cryptographic algorithms process data as large binary numbers. **AES** operates on 128-bit blocks (16 bytes = 32 hex characters). Hash functions like **SHA-256** produce fixed-length binary digests, universally displayed as hexadecimal.

```
SHA-256 HASH EXAMPLE (STRING: "HELLO")
```

```
2CF24DBA5FB0A30E26E83B2AC5B9E29E1B161E5C1FA7425E73043362938B9824
```

- *This 64-character hex string represents a 256-bit (32-byte) binary value. Every byte of the hash is two hex digits.*



Thank You!